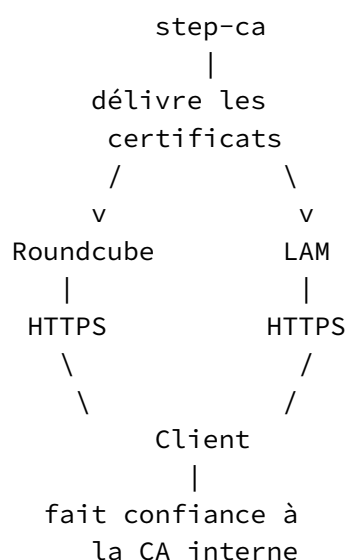

Déployer une autorité de certification et sécuriser les services

L'objectif est de mettre en place une autorité de certification interne avec **Smallstep step-ca**, puis de l'utiliser pour sécuriser les services Web existants de l'infrastructure.

Vous allez :

1. déployer une autorité de certification interne ;
2. configurer le client `step` pour utiliser cette autorité ;
3. générer des certificats pour Roundcube et LAM ;
4. ajouter les certificats aux services existants ;
5. installer la chaîne de confiance sur un poste client ;
6. vérifier les certificats et les connexions HTTPS.

L'architecture cible est :



Partie 1 — Créer l'autorité de certification

1. Préparer le projet

Créez un répertoire pour l'autorité de certification :

```
mkdir -p ~/on-premise/ca  
cd ~/on-premise/ca
```

Créez l'arborescence suivante :

```
ca/  
|-- compose.yaml  
|-- data/
```

2. Définir les noms des services

Dans la suite de l'exercice, nous utiliserons :

```
ca.embedded.test  
mail.embedded.test  
ldap.embedded.test
```

avec :

```
ca.embedded.test      -> step-ca  
mail.embedded.test    -> Roundcube  
ldap.embedded.test    -> LAM
```

Ces noms doivent pouvoir être résolus par les machines qui utilisent les services.

Dans l'environnement de TP, vous pouvez par exemple ajouter temporairement les entrées nécessaires dans `/etc/hosts` :

```
192.168.1.10 ca.embedded.test  
192.168.1.10 mail.embedded.test  
192.168.1.10 ldap.embedded.test
```

Adaptez l'adresse IP à votre infrastructure.

Vérifiez par exemple :

```
getent hosts mail.embedded.test
```

3. Créer le Docker Compose

Créez `compose.yml` :

```
services:

  step-ca:
    image: smallstep/step-ca:latest
    container_name: step-ca
    hostname: ca.embedded.test

    ports:
      - "9000:9000"

    volumes:
      - ./data:/home/step

    restart: unless-stopped

networks:
  default:
    external: true
    name: infrastructure
```

Le réseau `infrastructure` est le réseau utilisé par les autres services.

S'il n'existe pas encore :

```
docker network create infrastructure
```

4. Initialiser l'autorité de certification

Initialisez la PKI :

```
docker compose run --rm step-ca step ca init
```

Répondez aux questions.

Par exemple :

What would you like to name your new PKI?

Embedded Solutions CA

What DNS names or IP addresses would you like to add to your new CA?

ca.embedded.test

What address will your new CA listen at?

:9000

Définissez également un mot de passe permettant de protéger la clé de l'autorité.

Conservez ce mot de passe de manière sécurisée.

5. Identifier les éléments de la PKI

Examinez les fichiers générés :

```
find data -type f
```

Identifiez notamment :

```
certs/root_ca.crt  
certs/intermediate_ca.crt  
secrets/root_ca_key  
secrets/intermediate_ca_key  
config/ca.json
```

Identifiez lesquels de ces éléments :

- peuvent être distribués aux clients ;
- doivent rester privés ;
- sont nécessaires au fonctionnement de la CA.

La clé privée de l'autorité ne doit jamais être distribuée aux clients.

6. Démarrer step-ca

```
docker compose up -d
```

Vérifiez :

```
docker compose ps
```

Puis :

```
docker compose logs step-ca
```

7. Installer le client step

Installez le client Smallstep `step` sur la machine utilisée pour administrer la CA.

Vérifiez :

```
step version
```

8. Récupérer l'empreinte de la CA

Affichez l'empreinte du certificat racine :

```
step certificate fingerprint \  
data/certs/root_ca.crt
```

Vous obtenez une valeur similaire à :

```
aabbccddeeff...
```

Conservez cette empreinte.

9. Configurer le client step

Initialisez la confiance du client envers la nouvelle CA :

```
step ca bootstrap \  
  --ca-url https://ca.embedded.test:9000 \  
  --fingerprint aabbccddeeff...
```

Remplacez l'empreinte par celle obtenue précédemment.

Vérifiez ensuite l'accès à la CA :

```
step ca health
```

Résultat attendu :

```
ok
```

À ce stade, votre autorité de certification est opérationnelle.

Partie 2 — Générer les certificats des services

10. Préparer les répertoires

Créez :

```
mkdir -p ~/on-premise/certificates/roundcube  
mkdir -p ~/on-premise/certificates/lam
```

11. Générer le certificat Roundcube

Roundcube sera accessible avec le nom :

`mail.embedded.test`

Demandez un certificat à votre CA :

```
step ca certificate \
  mail.embedded.test \
  ~/on-premise/certificates/roundcube/mail.crt \
  ~/on-premise/certificates/roundcube/mail.key
```

Suivez les instructions affichées par step.

Vous obtenez :

`mail.crt`
`mail.key`

Le premier fichier contient le certificat.

Le second contient la clé privée associée.

12. Vérifier le certificat Roundcube

Inspectez-le :

```
step certificate inspect \
  ~/on-premise/certificates/roundcube/mail.crt
```

Vérifiez notamment :

- le sujet (Subject) ;
- l'autorité émettrice (Issuer) ;
- les dates de validité ;
- les Subject Alternative Names (SAN).

Vous devez retrouver :

`mail.embedded.test`

Vous pouvez également utiliser OpenSSL :

```
openssl x509 \  
  -in ~/on-premise/certificates/roundcube/mail.crt \  
  -noout \  
  -subject \  
  -issuer \  
  -dates \  
  -ext subjectAltName
```

13. Générer le certificat LAM

LAM sera accessible avec :

`ldap.embedded.test`

Générez son certificat :

```
step ca certificate \  
  ldap.embedded.test \  
  ~/on-premise/certificates/lam/ldap.crt \  
  ~/on-premise/certificates/lam/ldap.key
```

Inspectez-le :

```
step certificate inspect \  
  ~/on-premise/certificates/lam/ldap.crt
```

Le SAN doit notamment contenir :

`ldap.embedded.test`

14. Comparer les deux certificats

Utilisez :

```
openssl x509 \  
-in ~/on-premise/certificates/roundcube/mail.crt \  
-noout \  
-subject \  
-issuer
```

puis :

```
openssl x509 \  
-in ~/on-premise/certificates/lam/ldap.crt \  
-noout \  
-subject \  
-issuer
```

Répondez aux questions suivantes :

- Qu'est-ce qui est différent entre les deux certificats ?
 - Quel élément permet de déterminer pour quel service un certificat a été délivré ?
 - Quel élément permet de déterminer quelle autorité l'a délivré ?
 - Pourquoi ne faut-il pas utiliser le certificat de Roundcube pour LAM ?
-

Partie 3 — Ajouter le certificat au Roundcube existant

Vous disposez déjà d'un service Roundcube fonctionnel.

Conservez le déploiement réalisé précédemment. Ne remplacez pas Roundcube et ne modifiez pas son intégration avec Postfix et Dovecot.

L'objectif est uniquement d'ajouter HTTPS au service existant.

15. Vérifier Roundcube avant modification

Avant d'ajouter TLS :

```
docker compose ps
docker compose logs roundcube
```

Vérifiez que l'interface existante reste accessible :

`http://localhost:8080`

Connectez-vous avec un utilisateur.

Vérifiez :

- l'authentification ;
- la consultation d'un message ;
- l'envoi d'un message.

Ne commencez pas la modification tant que le service existant ne fonctionne pas.

16. Conserver la configuration existante

La configuration Roundcube réalisée précédemment doit être conservée.

En particulier, ne modifiez pas les paramètres fonctionnels :

```
environment:
  ROUNDCEMAIL_DEFAULT_HOST: dovecot
  ROUNDCEMAIL_DEFAULT_PORT: 143
  ROUNDCEMAIL_SMTP_SERVER: postfix
  ROUNDCEMAIL_SMTP_PORT: 25
  ROUNDCEMAIL_DB_TYPE: sqlite
```

L'ajout du certificat ne doit pas modifier les communications :

Roundcube → Dovecot
Roundcube → Postfix

17. Ajouter le certificat

Utilisez les fichiers générés précédemment :

```
mail.crt  
mail.key
```

Ajoutez-les au service Web utilisé par votre déploiement Roundcube.

Montez les fichiers en lecture seule.

Par exemple :

```
volumes:  
  - ./certificates/roundcube/mail.crt:/chemin/du/certificat/mail.crt:ro  
  - ./certificates/roundcube/mail.key:/chemin/de/la/cle/mail.key:ro
```

Adaptez les chemins à la configuration Web de votre déploiement existant.

Configurez ensuite le serveur Web pour utiliser :

```
mail.crt  
mail.key
```

et exposer Roundcube en HTTPS.

Ne stockez pas `mail.key` dans Git.

18. Redémarrer Roundcube

Après modification :

```
docker compose up -d
```

Vérifiez :

```
docker compose ps
```

Puis :

```
docker compose logs roundcube
```

Corrigez les éventuelles erreurs avant de poursuivre.

19. Vérifier le certificat présenté

Testez la connexion TLS :

```
openssl s_client \  
-connect mail.embedded.test:<PORT_HTTPS> \  
-servername mail.embedded.test
```

Examinez notamment :

```
subject  
issuer  
Verify return code
```

Le certificat présenté doit correspondre à :

```
mail.embedded.test
```

20. Tester explicitement avec la CA

Avant d'installer la CA dans le système :

```
curl \  
--cacert ~/on-premise/ca/data/certs/root_ca.crt \  
https://mail.embedded.test:<PORT_HTTPS>/
```

La page Roundcube doit être retournée sans erreur de validation du certificat.

N'utilisez pas :

```
curl -k
```

Cette option désactive la vérification du certificat et ne permet donc pas de valider le résultat de l'exercice.

21. Vérifier que Roundcube fonctionne toujours

Connectez-vous à Roundcube en HTTPS.

Vérifiez de nouveau :

- l'authentification ;
- la consultation des messages ;
- la réception d'un message ;
- l'envoi d'un message.

L'ajout de HTTPS ne doit pas avoir cassé les fonctionnalités déjà validées.

Partie 4 — Ajouter le certificat à LAM

Vous disposez également d'un déploiement LAM fonctionnel.

Comme pour Roundcube :

conservez le déploiement existant.

L'objectif est uniquement d'ajouter HTTPS.

22. Vérifier LAM avant modification

Vérifiez que LAM fonctionne avant d'ajouter le certificat.

Connectez-vous à l'interface et vérifiez que vous pouvez toujours consulter les informations de l'annuaire.

Conservez cette configuration fonctionnelle comme point de référence.

23. Ajouter le certificat LAM

Utilisez :

ldap.crt
ldap.key

Montez les fichiers dans le service Web utilisé par votre déploiement LAM.

Par exemple :

```
volumes:  
  - ./certificates/lam/ldap.crt:/chemin/du/certificat/ldap.crt:ro  
  - ./certificates/lam/ldap.key:/chemin/de/la/cle/ldap.key:ro
```

Configurez le serveur Web existant pour présenter ce certificat en HTTPS.

Ne modifiez pas la connexion entre LAM et OpenLDAP.

24. Vérifier le certificat présenté

Testez :

```
openssl s_client \  
  -connect ldap.embedded.test:<PORT_HTTPS> \  
  -servername ldap.embedded.test
```

Le certificat présenté doit correspondre à :

ldap.embedded.test

Puis :

```
curl \  
  --cacert ~/on-premise/ca/data/certs/root_ca.crt \  
  https://ldap.embedded.test:<PORT_HTTPS>/
```

LAM doit être accessible.

25. Vérifier le fonctionnement de LAM

Connectez-vous maintenant à LAM en HTTPS.

Vérifiez que les opérations réalisées précédemment fonctionnent toujours.

L'ajout de TLS ne doit pas modifier :

LAM → OpenLDAP

Partie 5 — Installer la CA sur le poste client

Pour le moment, vous avez explicitement indiqué le certificat racine à `curl` avec :

`--cacert`

L'objectif est maintenant de faire reconnaître l'autorité de certification par le système.

26. Installer le certificat racine

Sur Ubuntu :

```
sudo cp \
~/on-premise/ca/data/certs/root_ca.crt \
/usr/local/share/ca-certificates/embedded-solutions-ca.crt
```

Puis :

```
sudo update-ca-certificates
```

Le système doit indiquer qu'un nouveau certificat a été ajouté.

27. Vérifier sans `--cacert`

Testez Roundcube :

```
curl https://mail.embedded.test:<PORT_HTTPS>/
```

Puis LAM :

```
curl https://ldap.embedded.test:<PORT_HTTPS>/
```

Les connexions doivent maintenant fonctionner sans :

`--cacert`

et surtout sans :

`-k`

28. Vérifier avec OpenSSL

Pour Roundcube :

```
openssl s_client \  
-connect mail.embedded.test:<PORT_HTTPS> \  
-servername mail.embedded.test \  
-CAfile /etc/ssl/certs/ca-certificates.crt
```

Pour LAM :

```
openssl s_client \  
-connect ldap.embedded.test:<PORT_HTTPS> \  
-servername ldap.embedded.test \  
-CAfile /etc/ssl/certs/ca-certificates.crt
```

Résultat attendu :

Verify return code: 0 (ok)

Partie 6 — Vérification finale

Validez les éléments suivants :

Vérification	Résultat attendu
step-ca opérationnel	Oui
CA racine créée	Oui
Client step configuré	Oui
Certificat mail.embedded.test	Valide
Certificat ldap.embedded.test	Valide
SAN correspondant au service	Oui
Roundcube accessible en HTTPS	Oui
LAM accessible en HTTPS	Oui
CA installée sur le poste client	Oui
Chaîne de confiance validée	Oui
Fonctionnement initial des services conservé	Oui

Vous devez être capables d'expliquer la différence entre :

Le serveur possède un certificat.

et :

Le client fait confiance à l'autorité qui a délivré ce certificat.

Conservez dans votre documentation :

- la configuration de step-ca ;
- le certificat racine ;
- les certificats délivrés ;
- les modifications apportées à Roundcube ;
- les modifications apportées à LAM ;
- les commandes utilisées pour vérifier les certificats ;
- les résultats des tests.

Ne stockez pas les clés privées de la CA ou des services dans Git.

Diagnostic

step-ca ne démarre pas

```
docker compose ps
docker compose logs step-ca
```

Vérifiez ensuite :

```
step ca health
```

Le certificat ne correspond pas au service

Inspectez ses SAN :

```
openssl x509 \
  -in mail.crt \
  -noout \
  -ext subjectAltName
```

Le nom utilisé pour accéder au service doit être présent dans le certificat.

Le certificat est expiré ou n'est pas encore valide

```
openssl x509 \
  -in mail.crt \
  -noout \
  -dates
```

Le service ne démarre plus après l'ajout du certificat

Vérifiez en priorité :

- le chemin vers le certificat ;
- le chemin vers la clé privée ;
- les permissions ;
- la configuration du serveur Web.

Consultez ensuite :

```
docker compose logs
```

Le service fonctionne mais le certificat n'est pas reconnu

Testez explicitement avec la CA :

```
curl \
  --cacert root_ca.crt \
  https://mail.embedded.test:<PORT_HTTPS>/
```

Si cela fonctionne, mais que :

```
curl https://mail.embedded.test:<PORT_HTTPS>/
```

échoue, vérifiez l'installation de la CA dans le magasin de confiance du client.

Le navigateur refuse encore le certificat

Le navigateur peut utiliser un magasin de certificats différent de celui du système.

Vérifiez également :

- le nom DNS utilisé dans le navigateur ;
 - les SAN du certificat ;
 - la date de validité ;
-

-
- la chaîne de certification présentée.
-

Pour aller plus loin — Sécuriser IMAP et SMTP avec TLS

Roundcube est maintenant accessible en HTTPS.

Cependant, HTTPS protège la communication :

Navigateur → Roundcube

Il ne protège pas automatiquement les communications internes :

Roundcube → Dovecot

Roundcube → Postfix

Si vous avez terminé les activités précédentes, utilisez votre autorité `step-ca` pour sécuriser également ces communications.

Ajouter TLS à Dovecot

Déterminez d'abord le nom DNS avec lequel Roundcube accède à Dovecot.

Générez ensuite un certificat correspondant, par exemple :

```
step ca certificate \
  dovecot.embedded.test \
  dovecot.crt \
  dovecot.key
```

Inspectez-le :

```
step certificate inspect dovecot.crt
```

Configurez ensuite votre Dovecot existant pour :

- utiliser ce certificat ;
- utiliser la clé privée correspondante ;
- proposer IMAP avec TLS ;
- permettre au client de vérifier la chaîne de confiance.

Testez :

```
openssl s_client \  
-connect dovecot.embedded.test:993 \  
-servername dovecot.embedded.test
```

Le certificat présenté doit être celui délivré par votre step-ca.

Ajouter TLS à Postfix

Déterminez le nom DNS utilisé pour joindre Postfix.

Générez son certificat, par exemple :

```
step ca certificate \  
postfix.embedded.test \  
postfix.crt \  
postfix.key
```

Configurez ensuite votre Postfix existant pour utiliser :

```
postfix.crt  
postfix.key
```

Testez STARTTLS :

```
openssl s_client \  
-starttls smtp \  
-connect postfix.embedded.test:25 \  
-servername postfix.embedded.test
```

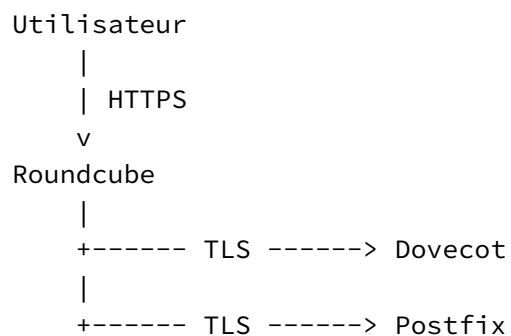
Vérifiez :

- le certificat présenté ;
 - son émetteur ;
 - son SAN ;
 - la chaîne de confiance.
-

Adapter Roundcube

Une fois TLS fonctionnel sur les deux services, adaptez votre configuration Roundcube afin d'utiliser les connexions sécurisées.

L'objectif est :



Vérifiez ensuite que :

- l'utilisateur peut toujours se connecter à Roundcube ;
- les messages peuvent toujours être consultés ;
- les messages peuvent toujours être envoyés ;
- les certificats sont délivrés par votre CA ;
- les noms utilisés correspondent aux SAN ;
- la vérification des certificats est active.

N'utilisez pas d'option désactivant la vérification des certificats pour faire fonctionner la configuration.